

From Characters to Transformers: lecture outline

Status: working source of truth for the Typst mega-deck

Format: 3 × 80 minutes, one continuous deck

Current deck: 170 conceptual frames; 178 handout pages including dividers and references; 436 progressive presentation pages.

The early Lecture 1 and Lecture 3 tables retain their original planning IDs. The revised Lecture 2 continuation below uses actual handout page numbers. Handout pages 60–81 introduce attention using bank prefixes; pages 82–88 calculate the incoming message at bank. Pages 89–116 finish that same example: projections, scaled attention, residual addition, prediction, generation, loss, autograd, a parameter update, and parallel causal training. Lecture 3 begins at page 117. Optional calculations follow it.

What students will learn

Students will build a name generator, identify what limits its context window, and work through attention and a complete Transformer. Both models learn to:

predict the next token from previous tokens

What changes is the architecture used to represent the context:

$\text{aabid} \rightarrow \text{fixed-window MLP} \rightarrow \text{adaptive retrieval} \rightarrow \text{self-attention} \rightarrow \text{multi-head attention} \rightarrow \text{Transformer} .$

Follow one small example. Write out its training rows, look up its vectors, check the shapes, and calculate an output. Ask students for the next step before revealing it. After introducing a general equation, return to the example and point to what each symbol means.

Worked example carried throughout

With context length three and one end-of-sequence prediction, aabid creates six examples:

$--- \rightarrow a, \quad --a \rightarrow a, \quad -aa \rightarrow b, \quad aab \rightarrow i, \quad abi \rightarrow d, \quad bid \rightarrow -.$

Style and notation

Teaching style

Use the step-by-step teaching method in the original 23-page handwritten next-token lecture. Styling comes directly from `common/metropolis.typ`, the theme used by the earlier course decks. Typst handles the fonts, spacing, dark-teal title bar, orange accent, and bottom slide number; there is no separate CSS file or local copy of the theme.

- a plain light background, dark text, and space around the worked example;
- one held drawing is redrawn in the same location while one new annotation, row, arrow, number, or highlight appears per build;
- headings are short questions, concrete statements, or operation names;
- simple tables, axes, arrows, matrices, and network sketches are the main visual vocabulary;
- consistent colors carry token identity across lookup, concatenation, and the network; red marks losses and forbidden attention;
- boxes exist only when the idea itself is a box, token, table, or model layer; avoid generic dashboard cards and dense UI grids;
- small exact numbers appear early; compact tensor notation arrives only after students can point to the corresponding marks in the held drawing;
- ask a question, draw the example, add one operation, calculate its result, and explain the result on the same drawing;
- whenever an operation is easier to see as a network, draw the neurons, arrows, residual paths, or attention routes rather than replacing them with a paragraph;
- slide code is limited to a short PyTorch fragment, normally two to four lines. The complete executable path lives in the runnable companion.
- text-free editorial illustrations may support a conceptual hook; all labels, equations, arrows, and technical diagrams remain native Typst vectors.

Use native Typst/Fletcher for technical diagrams. Keep the reasoning and reveals close to Nipun's notes, with readable type rather than a handwriting font.

- Teal: observed context, token embeddings, keys.
- Blue: model-produced scores, queries, probabilities.
- Orange: targets, values, retrieved information.
- Green: correct/retained paths and residual additions.

- Red: loss, invalid future positions, failure modes.
- Ink/cream: conventions, shapes, and unchanged infrastructure.
- Rows always mean tokens/queries; columns in an attention map always mean candidate source tokens/keys.
- The persistent numerical case in Lecture 2 is She deposited money in the bank .. Treat words and punctuation as tokens, with eight vocabulary types. Input embeddings, queries, keys, values and messages are three-dimensional; vocabulary logits and probabilities are eight-dimensional. All numbers are checked by `mega/bank_training.py`; round only displayed results.

Canonical notation for Lectures 1 and 2

Before introducing Q/K/V, concatenate or average **lookup embeddings** e_j . Introduce v_j only when explaining the separate value role: information mixed using weights chosen by query-key matches. Preserve the embedding notation: $e_i \rightarrow e'_i = e_i + \Delta e_i$. Do not introduce a second single-token symbol before explaining what attention adds.

Symbol	Meaning and shape
t_i	Integer token ID at position i ; IDs are $0, \dots, \mathcal{V} - 1$. Code stores them in <code>ids</code> .
$E, e_i = E[t_i]$	Learned lookup table, $ \mathcal{V} \times d$; one embedding row, $1 \times d$. Literal token subscripts such as e_a differ from positional e_i .
$q_i = e_i W_Q, k_i = e_i W_K$	Initial query and key rows, each $1 \times d_k$; W_Q, W_K are learned $d \times d_k$ matrices.
$v_i = e_i W_V$	Initial value row, $1 \times d_v$; W_V is a learned $d \times d_v$ matrix.
$h_i = \sum_j \alpha_{ij} v_j$	$1 \times d_v$ incoming message specifically for query position i .
$\Delta e_i = h_i W_O$	$1 \times d$ contextual update; the learned output map W_O has shape $d_v \times d$.
$e'_i = e_i + \Delta e_i$	Context-enriched embedding of this occurrence. It does not overwrite the lookup row. Predict vocabulary logits from this row.
X	Sequence matrix, $T \times d$, initially stacking $e_i = E[t_i]$. Distinguish these sequence rows from the whole lookup table E .
Q, K, V, H	Corresponding row-stacked matrices: $T \times d_k, T \times d_k, T \times d_v, T \times d_v$.
$\Delta X, X'$	$T \times d$ matrices stacking Δe_i and e'_i ; matrix names introduce no new single-token symbol.
R, r_i	Learned position table and its $1 \times d$ row, introduced later. Math positions start at 1; Python position indices start at 0.
$e_i^{(0)} = e_i + r_i$	Once positions are introduced, this position-aware embedding is row i of X and supplies all three projections.
$e_i^{(1)} = e_i^{(0)} + \Delta e_i$	After the first contextual update with positions, this is row i of X' .

Use **row-vector multiplication throughout**. A contains attention weights α_{ij} , has shape $T \times T$, and gives $H = AV$. The complete residual path is $\Delta X = H W_O$, $X' = X + \Delta X$, then $Z = X' U + b$. Initially $W_O = I$ in the toy. Values are not merely renamed lookup embeddings. Before positions, setting $W_V = I$ makes $v_i = e_i$; after positions, it makes $v_i = e_i^{(0)}$ instead. In general, the value is a projection. The early row calculation is $e_i \rightarrow e'_i$; the later position-aware calculation is $e_i^{(0)} \rightarrow e_i^{(1)}$.

Use w for the fixed context length, d_h for MLP hidden width, and $|\mathcal{V}|$ for vocabulary size. L1's concatenated input is $a_0 = [e_1, \dots, e_w]$, and its hidden row is $a_1 = \sigma(a_0 W_1 + b_1)$. Here W_1 has shape $wd \times d_h$ and W_2 has shape $d_h \times |\mathcal{V}|$. The optimizer updates the embedding/position tables and learned matrices and biases; $X, Q, K, V, A, H, \Delta X, X'$ are recomputed on each forward pass.

Overall narrative

1. What can neural networks do with text?
2. Can generation be reduced to repeated classification?

3. Build the full fixed-window character model.
 4. Characters are only one possible tokenization.
 5. Generalize the same objective to natural-language tokens.
 6. Calculate how concatenation grows with context length.
 7. Try averaging and test what information it loses.
 8. Make the weights depend on the query.
 9. Name the three roles Query, Key, and Value; distinguish matching from sending information.
 10. Add the incoming contextual update to the embedding, predict, and generate with fixed parameters.
 11. View the loss calculation as a graph, use autograd to obtain gradients, update parameters, then introduce masked parallel training.
 12. Let several retrieval mechanisms operate in parallel: MHA.
 13. Combine MHA with the MLP and residual ideas students already know.
 14. Stack blocks and attach the same next-token objective.
 15. Compare the two models on the same character prediction task.
-

Lecture 1: Next-token prediction

Timing: ~80 minutes. Actual deck: 51 conceptual frames. The base planning map below is frames 1–50, plus the inserted loss-code frame listed above.

Act 1 — Why language? (frames 1–12)

#	Teaching frame	Content, worked example, and visual direction
1	Deep learning for language	Almost empty opening: “How can a neural network work with language?” Reveal a native-vector collage of email→spam, review→sentiment, article→summary, English→Hindi, context+question→answer, prompt→continuation. No equations.
2	One input → one label	Review: “The acting was wonderful but the ending was disappointing.” Show a binary softmax with 0.61/0.39. Ask whether this is fundamentally different from image classification.
3	Intent classification	Three cards: book a Delhi flight→BOOK_FLIGHT; account balance→CHECK_BALANCE; cancel booking→CANCEL_BOOKING. Conclude text → one label.
4	Token-level prediction: named entities	“Sundar Pichai visited IIT Gandhinagar on Monday.” Reveal aligned PERSON/O/ORG/DATE tags. Establish sequence→aligned sequence.
5	Translation changes the sequence length	English sentence enters a model and exits as Hindi. Ask whether token counts must match. Establish sequence→different-length sequence.
6	Summarize a notice	Compress a six-line IITGN-style notice to one sentence. Leave the question “How does the model decide what matters?” unanswered; this phrase returns in attention.
7	Answer a question using the context	Context about IITGN’s Palaj campus, question “Where?”, answer “Palaj.” Seed the idea that useful context depends on the question.
8	Represent similar meanings with nearby vectors	Put “student solved assignment” close to “learner completed homework” and far from a rain sentence in a small 2-D vector map.
9	Generation has many valid answers	Prompt: “The student opened the laptop and ...” Collect several plausible continuations. Ask whether there is one correct answer.
10	Generation is still classification	Reveal a softmax over plausible next tokens. Link directly to categorical classification.
11	Generate a name	“Can we generate Indian names?” Reveal a list from aabid to zee1.
12	What exactly should we predict?	Character, word, or something between? Name the general unit “token,” then choose one character for the first complete model. Defer BPE.

Act 2: Build a character model (frames 13-34)

#	Teaching frame	Content, worked example, and visual direction
13	The task: generate an Indian name	Generate Indian names; vocabulary $a...z$ plus one boundary token $-$; $ V = 27$. Clarify that the toy $-$ plays BOS/padding/EOS roles only for this derivation.
14	Generation is repeated 27-class classification	$a \ a \ b \rightarrow ?$; reveal a 27-class probability strip with i high. State $f_{\theta}(a, a, b) = p(c_{\text{next}} a, a, b)$.
15	Choose a context length	Commit to $w=3$; define the probability of t_i given IDs $t_{i-3:i-1}$. Do not criticize it yet.
16	Construct the training examples	Progressive reveal of the six exact context-target rows.
17	Slide the context window	Animate a width-three sliding window across $---aabid-$. Add a “remember this” bookmark for causal parallel training later.
18	Repeat for every name	A name of length L yields $L+1$ targets including EOS; total examples are $\text{sum}_n(L_n+1)$.
19	How do we turn characters into numbers?	Put $a \ b \ i$ beside the MLP’s need for a row vector of numbers. Invite one-hot as the obvious first answer.
20	One-hot encoding	Show three one-hot rows and equal zero dot products. Ask what similarity they encode.
21	Learn an embedding for each character	Introduce $E \in \mathbb{R}^{(27 \times d)}$ and choose $d=2$; highlight one row per character.
22	Look up the embedding	Map $a, b, i \rightarrow \text{IDs} \rightarrow \text{explicit rows}$, using the same colors from token through table to vector.
23	Check the shapes	$\text{ids}=(t_1, t_2, t_3)$ has shape $(3,)$, $E[\text{ids}]$ has shape 3×2 ; write the three-row matrix. Link positional $e_i = E[t_i]$ to the token-named lookup rows.
24	Concatenate the embeddings	Flatten the three rows into $a_0=[e_1, e_2, e_3]$, shape 1×6 . Keep position-colored segments visible.
25	Why concatenate?	Compare a, b, i with i, b, a ; concatenation preserves slots while a plain sum does not. Plant the need for a more scalable representation of position.
26	Pass the vector through an MLP	Show $a_1 = \sigma(a_0 W_1 + b_1)$ and $z = a_1 W_2 + b_2$. Use $W_1: w_d \times d_h$, $W_2: d_h \times 27$; explain right-multiplication of row vectors.
27	Why exactly 27 outputs?	One output logit per possible next character; conclude this is 27-class classification.
28	Turn the 27 scores into probabilities	Give a few toy logits, normalize them, and draw a probability bar chart with target i .
29	Compute the loss	Compare $-\log .8 \approx .223$ with $-\log .01 \approx 4.605$. The correct character should receive probability mass.
30	Learn the embeddings and MLP weights	Box the embedding table and both MLP layers. Emphasize that backprop learns the representation and classifier jointly.
31	When could two embeddings become similar?	Similar context produces related gradient pressure; show an explicitly illustrative 2-D character embedding, without claiming human-like character semantics.
32	Sample, append, repeat	Start from $---$, sample a , shift to $--a$, and continue until EOS to generate $aabid$.
33	Training and generation	Two-column comparison: known target/loss/update/many windows versus unknown target/choose/append/autoregressive loop.
34	Our character language model	Probability of t_i given $t_{i-3:i-1}$. It is tiny and character-level, but it predicts the next token from previous tokens.

Act 3 — What should a token be? (frames 35–44)

#	Teaching frame	Content, worked example, and visual direction
35	Why begin with characters?	They made the first model inspectable; show how long “deep learning is amazing” becomes character by character.
36	Character tokens	transformer -> $t x ...$; tiny vocabulary, long sequence, almost no unknown strings.
37	Word tokens	Natural short sequence, but walk/walks/walked/walking/walker/walkability expands vocabulary.
38	What happens to an unseen word?	A test string such as hyperhappiness collapses to <UNK> under a word vocabulary.
39	Can we split text at spaces?	Use “IITGN में आज transformer lecture है :)”, plus a URL and code fragment. Ask what a word is across scripts, punctuation, emoji, and code.
40	Subword tokens	Split unbelievable, tokenization, and a rare scientific word into reusable chunks. Keep BPE mechanics for a later lecture.
41	Choose the token size	Compare vocabulary size $ \mathcal{V} $ and length T : tiny vocabulary/long sequence for characters; moderate subwords; huge vocabulary/short sequence for words.
42	Compare the trade-off	Table: vocabulary size, sequence length, OOV behavior, semantic convenience. Highlight that attention cost depends strongly on T .
43	Tokens become integer IDs	Small vocabulary table for “The cat sat on the mat” and the corresponding ID sequence.
44	Use token IDs in the model	Define $t_1, \dots, t_T$, with zero-based IDs $t_i \in \{0, \dots, \mathcal{V} - 1\}$. Display whole words for readability.

Act 4: Predict natural-language tokens (frames 45-50)

#	Teaching frame	Content, worked example, and visual direction
45	Change the tokens, keep the model	Place a a b -> i beside cat sat on -> the; identical embedding→concatenate→MLP→softmax pipeline.
46	What could come next?	“The cat sat on the ____.” Gather mat/floor/chair/sofa, then show a distribution instead of one truth.
47	Construct examples from a sentence	deep -> learning, deep learning -> is, Relate prefixes to the earlier sliding windows.
48	Multiply the next-token probabilities	Work $p(\text{deep learning is fun})$ into the chain-rule product, one factor at a time.
49	How should we represent the context?	Sum negative log-probabilities of $t_{\{i+1\}}$ given $t_{\{1:i\}}$. The objective is settled; context representation is the open problem.
50	What if the context keeps growing?	aab -> i beside The cat sat on the -> mat; ask “How should we represent arbitrarily long context?”

Lecture 2: How attention works

Timing target: ~80 minutes. Actual deck: 62 conceptual frames (handout 55–116). Use the bank example throughout. Start with two prefixes to make context visible, then retain only She deposited money in the bank for arithmetic. The river prefix is an illustration of desired contextual behaviour, not a second numerical model or a claim about a trained result.

Keep the distinction visible: **lookup embedding** → **embedding + incoming contextual update** → **prediction**. Before attention, average e_j ; only introduce v_j when the separate information-sending role is needed. The three jobs are questions that guide our interpretation, not literal English questions computed by the model.

Act 5: Why the context needs a different representation (pages 55–70)

Hand- out page	Teaching frame	What students see or answer
55	The fixed-window pipeline	Recall the L1 token→lookup→concatenation→MLP→softmax model. This is the only brief return to the character example.
56	What if the window is five tokens?	Concatenation gives $5d$ coordinates. It still works.
57	What if the window is 100 tokens?	Input width and first-layer parameter count grow.
58	What if the window is 10,000 tokens?	Expose the architectural cost of concatenating every slot.
59	The window is part of the architecture	W_1 has shape $wd \times d_h$: a different window changes the network's input layer.
60	Which context would help after bank?	Compare She deposited money in the bank ___ and She watched water by the bank _____. Both end at position 6. Bank is known; the token after it is unknown.
61	The useful clue may lie outside the window	Enrich the finance prefix with after a long wait near the bank. A last-three-token window misses money. Distance alone is not relevance.
62	Context cannot supply a clue that is missing	She waited near the bank ___ does not settle finance versus river. The mechanism uses available evidence; it does not create missing facts.
63	Same lookup embedding, different context	Both prefixes use the same $E[t_6] = e_{\text{bank}}$. Different source words should inform different contextual representations. Do not invent learned-coordinate meanings.
64	Keep the embedding; add context	Introduce $e'_6 = e_6 + \Delta e_6$. The predictor reads this contextual embedding. Context addition does not replace the embedding table.
65	What should context add to bank's embedding?	Start with the toy $e_6 = (1, 1, 0)$ and reveal three unknown update coordinates. Add matching coordinates to form e'_6 . Ask how the six input embeddings supply those three numbers; the next slide tries their average.
66	Average the six input embeddings	Reveal all six 3D lookup rows and calculate $\Delta e_6 = (4, 2, 2)/6 = (2/3, 1/3, 1/3)$. This exact table returns later.
67	Problem 1: equal weights cannot select a clue	Every source gets $1/6$; money and the receive the same multiplier.
68	Problem 2: an average forgets order	Maya told Ravi about the bank versus Ravi told Maya about the bank: same token multiset and final bank, different speaker. Both the mean message and retained last lookup row are identical. Defer positions while first solving selective weighting.
69	Give useful sources more weight	$\Delta e_6 = \sum_j \alpha_{6j} e_j$, nonnegative weights summing to one. One weight multiplies every coordinate of a source embedding. Introduce the message symbol h_6 only after the separate value role.
70	The receiving position needs a say	Three short alternatives: distance alone, fixed word importance, or compatibility between the receiving position's need and a source's matching features.

Act 6: Query/key matching and value messages (pages 71–88)

Hand- out page	Teaching frame	What students see or answer
71	What information would help after bank?	Bank is already known; the next token is not. Consider possible topic, action, or person requests, with money, deposited, and She as source clues. Choose one toy financial-clue request for the calculation. These English questions are interpretations, not literal questions computed by a model.
72	Turn bank's embedding into a query	Start from $e_6 = (1, 1, 0)$ and specify $W_Q = \text{diag}(1, 0, 0)$. Expand each coordinate product to obtain $q_6 = (1, 0, 0)$. Keep the shapes $(1 \times 3)(3 \times 3) \rightarrow 1 \times 3$ visible and match them to three lines of PyTorch.
73	A key describes what a source can match	Form $k_j = e_j W_K$ with the toy choice $W_K = I$. Highlight money, the, and bank as three candidate sources. Their keys describe matching features; the value role has not yet been introduced. Identity is a teaching initialization, not a required architecture.
74	Match the request, not another copy of bank	Compare the same q_6 with the highlighted keys: money scores 2, the scores 0, and bank scores 1. The query asks for a kind of clue, not a duplicate embedding; self-match need not win. These are scores, not probabilities. The later softmax includes all six sources.
75	The same query can find different evidence	Return to the finance and river prefixes. The same bank lookup row gives the same first-layer query under the same W_Q , while the different source words provide different keys. Keep this contrast qualitative: do not invent water coordinates or claim the toy has learned word meanings.
76	Value: what should this source send?	One source, two jobs. Bank's input $(1, 1, 0)$ yields key $(1, 1, 0)$ and value $(1, 2, 0)$. Introduce v as a transformed information vector, not a renamed lookup embedding.
77	Match queries and keys; mix the values	Hold a native diagram: query+keys→scores→softmax→weights; weights+corresponding values→message. The weight found with k_3 multiplies v_3 .
78	Turn the collected message into an embedding update	$h_6 = \sum_j \alpha_{6j} v_j$. Six source weights give one three-coordinate message, not a selected word or vocabulary probability. Map it into Δe_6 , add it to e_6 , then predict.
79	Same matches can carry different messages	Controlled intervention: hold inputs, queries and keys fixed. Change value coordinate scaling from $(1, 2, 1)$ to $(0, 2, 1)$. Scores and weights do not change; message coordinate 1 becomes zero. Retained e_6 survives. This is neither a word replacement nor SGD.
80	A contextual update is not a learning update	Computing e'_6 changes this occurrence's representation, not E . Training changes shared parameters for later forward passes.
81	Our bank model will predict after adding context	Name $\Delta e_6 = h_6 W_O$, initially $W_O = I$. Restore the original value transformation before numerical work. This is a small residual-attention model; the full block comes in L3.

Work out the message with all six sources (pages 82–88)

Hand- out page	Teaching frame	What students see or answer
82	Keep the query; now inspect all six sources	Keep the already-computed $q_6 = (1, 0, 0)$. Show all six known input words with explicit positions and IDs, then inspect every candidate key. This extends the three-source comparison rather than restarting the query introduction.
83	Give each input a key and a value	A row-aligned table of all six 3D keys and values. Keep each key/value pair attached to its source position.
84	Compare the same query with all six keys	Expand all dot products to obtain $(0, 1, 2, 0, 0, 1)$. Bank's query and key differ; self-match need not win. These warm-up scores are unscaled.
85	Exponentiate the six match scores	Hide key details; reveal $\exp(0), \exp(1), \exp(2)$ and amounts 1.000, 2.718, 7.389 in source order. Round first for this hand calculation; displayed amounts total 15.825. Use $\exp$ so the exponential function is distinct from embedding e_i .
86	Divide every amount by the same total	Separate amount, division, denominator and weight columns. Weights $(.063, .172, .467, .063, .063, .172)$ sum to 1. State that α_j abbreviates α_{6j} while query 6 is fixed.
87	Use each weight to mix the corresponding value	Six values feed h_6 through weighted arrows. One weight multiplies every coordinate of its value.
88	Calculate the three coordinates of the message	Reveal all six contributions to each coordinate. Rounded-weight arithmetic yields $h_6 = (1.278, .470, .235)$. This is incoming information, not the final token state.

Act 7: Finish one prediction, then generate (pages 89–103)

All token-level vectors use row-vector notation. Embeddings and Q/K/V/messages have three coordinates; vocabulary rows have eight, ordered as *She*, *deposited*, *money*, *in*, *the*, *bank*, *.*, and. Matrices are specified, not pretrained. The warm-up uses explicit rounding; the trainable model uses float64 without intermediate rounding.

Hand- out page	Teaching frame	What students work through
89	Keep bank, then add what the context contributes	First addition: $(1, 1, 0) + (1.278, .470, .235) = (2.278, 1.470, .235)$. If the message is zero, the original input remains. This forward computation does not overwrite E .
90	Start with one embedding row per token	All eight E rows and six input rows of X ; identify bank's token ID 5 at position 6. Two-line lookup code.
91	Three learned matrices give one vector three jobs	W_Q, W_K, W_V are shared trainable transformations; q, k, v are their recomputed outputs. Shapes $d \times d_k, d \times d_v$.
92	Use the same query map at every position	Apply the same $W_Q = \text{diag}(1, 0, 0)$ to She, money, and bank: $q_1 = (0, 0, 0)$, $q_3 = (2, 0, 0)$, and $q_6 = (1, 0, 0)$. Stack the rows as $Q = XW_Q$, shape 6×3 , with two lines of code. This generalizes the earlier bank multiplication rather than repeating it.
93	The other two projections reproduce our keys and values	$W_K = I, W_V = \text{diag}(1, 2, 1)$ reproduce the full six-row tables. Their diagonal forms are initializations, not constraints.
94	Is the value just the original embedding?	Before positions, $W_V = I$ makes $v_i = e_i$. Usually W_V learns what features to send. Matching width and value width can differ.
95	Map the incoming message into the space we are updating	$W_O : d_v \times d$ maps h into Δe before addition. Initially identity, but trainable. Distinguish message, mapped update, and contextual embedding.
96	Why divide a dot product by the square root of its width?	Under independent zero-mean/unit-variance coordinates, dot-product variance grows as d_k and standard deviation as $\sqrt{d_k}$. Compare widths 4 and 64. Scaling avoids width-driven softmax sharpness; it is not cosine normalization.
97	Use scaled scores for the rest of the model	Divide by $\sqrt{3}$, recompute attention and $h_6 = (1.017777, .571364, .285682)$. These replace the warm-up numbers.
98	Predict from bank plus its contextual update	$e'_6 = (2.017777, 1.571364, .285682)$. Specify all of $U : 3 \times 8$ and b ; derive logits 2.018 for period, 1.571 for and, zero for the other six.
99	Vocabulary softmax answers "what could come next?"	Derive $p(.) = .410236$ and $p(\text{and}) = .262518$. Compare attention softmax over six source positions with vocabulary softmax over eight token types.
100	Follow one complete forward pass	Lookup→Q/K/V→scaled matches→weights→message→mapped addition→logits. Keep the tensor shape and the inspected bank row beside every operation.
101	Generate by choosing one token from this distribution	Greedy choice is period; an explicitly hypothetical sample is and. Append without changing parameters.
102	The appended token supplies the next query	$e_7 = (.5, 1, 1)$ gives $q_7 = (.5, 0, 0)$. Seven keys/values, same learned matrices.
103	Reuse the same model for every generation step	$h_7 = (.791268, .811055, .405528)$; updated state $(1.291268, 1.811055, 1.405528)$. Derive the new distribution: $p(.) = .230883$, $p(\text{and}) = .388269$. Repeat.

Act 8: Learn from the observed next token (pages 104–109)

The main lecture explains the backward pass with the computational graph and autograd. Manual derivatives stay in the appendix and executable companion.

Hand-out page	Teaching frame	What students work through
104	The observed next token tells us the prediction error	Restore the six-token prefix and observed period target. Cross-entropy $-\log(.410236) = .891023$. Target is used for loss, not as the query.
105	Which numbers will the optimizer change?	Parameters: $E, W_Q, W_K, W_V, W_O, U, b$. Intermediates: inputs, Q/K/V, scores, weights, messages, contextual updates and predictions. State update and parameter learning are different.
106	The forward pass is a computational graph	Include every parameter input, the value path, and the retained-input bypass. The two paths meet at addition before the vocabulary head and loss.
107	Autograd follows the graph backward	Reverse arrows through the head, addition, W_O , values and attention weights, projections and embedding table. <code>loss.backward()</code> accumulates gradients.
108	Apply the update to all learned parameters	Zero gradients, forward/loss, backward, SGD step. Every matrix entry can change, including W_O ; activations are recomputed, not optimized directly.
109	Run the same prefix again after the update	Verified $p(\cdot)$ changes $.410236 \rightarrow .657754$; loss $.891023 \rightarrow .418925$. One-example improvement is not generalization.

Act 9: Train all positions without reading ahead (pages 110–116)

Hand-out page	Teaching frame	What students work through
110	Predict every next token without reading ahead	Shift targets by one. At money, mask future in/the/bank before softmax; forbidden weights are zero. Introduce masking before full parallel matrices.
111	Give each position its own vector	Introduce $e_i^{(0)} = e_i + r_i$, now row i of X . Show one illustrative nonzero position vector, but reset the numerical experiment to original parameters and $R = 0$. Q/K/V use $e_i^{(0)}$; the contextual output is $e_i^{(1)}$. A plain mean of token+position rows still loses order.
112	Compute all six query rows together	$Q, K, V : 6 \times 3$, masked $A : 6 \times 6$, $H : 6 \times 3$, $X' = X + HW_O$, $Z : 6 \times 8$. Name each normalization axis.
113	The matrix pass contains our earlier calculation	Display actual A and H . Row 3 has zero future weights; row 6 recovers the earlier message and residual state.
114	The complete forward pass in PyTorch	Three short code chunks: embedding/position/projections, scaling/masking, mixing/residual/head.
115	Average the six losses, then update the parameters	Restart initial parameters plus $R = 0$. Update all eight parameter tensors. Mean loss $2.339914 \rightarrow 2.199261$; last-row loss rises $.891023 \rightarrow .947535$, illustrating the aggregate objective.
116	One complete next-token model	Training changes shared parameters; generation grows the prefix with parameters fixed. Bridge to multiple heads, normalization and the position-wise MLP.

Optional appendix (pages 173–175): softmax-mixture derivative, full matrix backward pass including the residual bypass and W_O , and dot-product direction/magnitude intuition.

Questions to ask before revealing the next build

- Same bank lookup row: must the contextual representation stay the same?
- Before choosing the next token, which earlier word could supply a topic, action, or person clue?
- What changes between e_6 and $q_6 = e_6 W_Q$, and what are the three shapes in that multiplication?
- Why can money match this request more strongly than bank itself?
- If bank's first-layer query stays the same in two prefixes, what different evidence can it inspect?
- A source matches strongly: does that tell us what vector it sends?
- If only W_V changes, which tensors can change and which cannot?
- If the incoming message is zero, what reaches the predictor?

- Did we update the shared embedding table while reading this one prefix?
- At generation step 2, where does the new query come from?
- Why are six attention weights different from eight next-token probabilities?

Lecture 3: Build a Transformer

Timing: ~80 minutes. Actual deck: 54 conceptual frames. The base planning map below is frames 105–154, plus the four Lecture 3 inserts listed above.

Act 10 – Multi-head attention (frames 105–116)

#	Teaching frame	Content, worked example, and visual direction
105	Recall one attention head	One query space, one key space, one value space, one weighted retrieval per token.
106	Why use more than one head?	A token may simultaneously need local syntax, entity identity, and long-range topic information.
107	Give each token several attention heads	Three color-coded heads: “nearest modifier?”, “which entity?”, “what topic?” Avoid claiming these are guaranteed learned semantics.
108	Each head learns its own projections	$Q_h = XW_Q^h$, $K_h = XW_K^h$, $V_h = XW_V^h$.
109	One head may look nearby	Local/previous-token weights on a short sentence. Keep the same row/column convention.
110	Another head may look back to a person	Long-range noun/pronoun or subject/verb relation on the same sentence.
111	A third head may notice a boundary	Copy/entity or delimiter behavior on the same tokens. Explicitly label maps as pedagogical possibilities.
112	The heads do the same calculation separately	$\text{head}_h = \text{Attention}(Q_h, K_h, V_h)$ with shapes $T \times d_v$.
113	Put the head outputs side by side	$\text{Concat}(\text{head}_1, \dots, \text{head}_H)$ restores a width of $H \cdot d_v$.
114	Mix the concatenated features with a linear layer	Output projection returns to model width d_{model} .
115	Check the widths before moving on	Work the common case $d_k = d_v = d_{\text{model}}/H$; show why concatenation has width d_{model} .
116	What can an attention map tell us?	Heads are learned distributed computations; visual patterns can be inspected but should not be over-narrated.

Act 11 — Build one Transformer block from familiar parts (frames 117–129)

#	Teaching frame	Content, worked example, and visual direction
117	Attention mixes rows; the FFN mixes features	Contrast information exchange between token rows with nonlinear feature recombination inside each row.
118	Give every token the same small neural network	Position-wise FFN: $\text{FFN}(x) = W_2 \cdot \text{phi}(W_1 x + b_1) + b_2$. Same weights at every token.
119	Across tokens, then within a token	Two-axis grid visual distinguishes across-position mixing from within-position feature transformation.
120	Why does the FFN widen in the middle?	$d_{\text{model}} \rightarrow d_{\text{ff}} \rightarrow d_{\text{model}}$; intermediate width creates nonlinear feature combinations.
121	Bring back the ResNet shortcut	Ask what happens if a new sublayer is initially unhelpful. Residual paths preserve and refine representations.
122	Add the attention output to its input	$\mathbf{x} = \mathbf{x} + \text{MHA}(\mathbf{x})$; draw an uninterrupted residual stream with a branch through attention.
123	Add the FFN output to its input	$\mathbf{y} = \mathbf{x} + \text{FFN}(\mathbf{x})$; repeat the same visual grammar.
124	LayerNorm works across one token's features	Work $[1, 3, 5] \rightarrow [-2, 0, 2] \rightarrow [-1.22, 0, 1.22]$, then name the general per-row operation.
125	Where should LayerNorm go?	Show both equations; choose modern pre-norm for the remainder and mark the architectural convention explicitly.
126	Put the sublayers together	$\mathbf{x} \rightarrow \text{LN} \rightarrow \text{causal MHA} \rightarrow + \rightarrow \text{LN} \rightarrow \text{FFN} \rightarrow +$. One centerpiece native-vector diagram.
127	Write the block as two updates	$\mathbf{u} = \mathbf{x} + \text{MHA}(\text{LN}(\mathbf{x}))$; $\mathbf{y} = \mathbf{u} + \text{FFN}(\text{LN}(\mathbf{u}))$. Match every equation color to the diagram.
128	Stack several blocks	Repetition deepens the sequence of retrieval and transformation; parameters differ by layer.
129	The next block uses the updated token states	One layer can route directly; several layers can retrieve, transform, and retrieve again. Avoid promising human-readable reasoning steps.

Act 12 — The complete decoder-only language model (frames 130–142)

#	Teaching frame	Content, worked example, and visual direction
130	Begin with token and position information	Token IDs→token embeddings + positional information; resulting matrix is $T \times d_{\text{model}}$.
131	Turn the final token state into token scores	Final state at each position→linear projection to v logits→softmax.
132	The decoder-only model from end to end	Tokenizer→embedding+position→ $L \times$ decoder blocks→LM head→next-token distributions.
133	Slide the targets one place to the left	Align <code><BOS></code> deep learning is with targets deep learning is fun; no hand-waving about which position predicts what.
134	During training, we know the whole sentence	During training, each row receives the true prefix under the mask; no sampled token is fed within that pass.
135	Score every next-token guess	$L = -\sum_t \log p(x_{t+1} x_{\leq t})$; optional padding mask excludes non-data positions.
136	The next-token objective stays the same	Place the aabid cross-entropy and the Transformer token loss in one equation frame.
137	Training: one pass, many next-token guesses	Whole sequences, parallel masked positions, loss, backprop, parameter update.
138	Generation: one new token, then go again	One current prefix, final-position distribution, choose/sample, append, repeat.
139	Generation, first step	Start <code><BOS></code> The; show attention flow and choose student.
140	Generation, one step later	Append and recompute/extend; choose opened. Keep probabilities visible.
141	Cache the keys and values from earlier positions	Past K/V rows do not change during autoregressive decoding; reuse them instead of recomputing every old projection.
142	What does a longer context cost?	Vanilla attention stores an $T \times T$ training map and decoding KV cache grows with T ; direct access has a real memory/compute cost.

Act 13: Compare the models (frames 143-154)

#	Teaching frame	Content, worked example, and visual direction
143	Three common ways to arrange Transformer blocks	Encoder-only, decoder-only, encoder–decoder cards; distinguish visibility and typical objective.
144	Encoder-only: read the whole input	Bidirectional self-attention for contextual representation/classification; BERT as an example, not a decoder.
145	Decoder-only: read only the prefix	Causal self-attention for next-token generation; GPT-style model is the deck’s main construction.
146	Encoder-decoder: read one sequence, write another	Source encoded bidirectionally, target decoded causally; useful for translation and conditioned generation.
147	Self-attention and cross-attention use the same calculation	Self: Q/K/V from same stream. Cross: queries from decoder, keys/values from encoded source.
148	The 2017 Transformer had an encoder and a decoder	One historical slide: encoder–decoder architecture from “Attention Is All You Need”; then return to the modern decoder-only focus.
149	Put the sequence models side by side	Table: fixed-window MLP, RNN, temporal convolution, self-attention—context flexibility, path length, training parallelism, and cost.
150	Longer memory has a price	Vanilla self-attention is $O(T^2)$ in pair scores; RNN is sequential; fixed MLP hard-codes k . No architecture wins every axis.
151	What does attention give us?	Gives learned content-dependent routing. Does not guarantee factuality, reasoning, interpretability, or unlimited memory.
152	Four quick checks	Four claims: “weights are explanations,” “MHA means one linguistic rule per head,” “masking is only used at generation,” “a Transformer removes the MLP.” Students correct each.
153	Compare our two language models	Side-by-side pipelines: fixed three-row concatenation and causal self-attention over an arbitrary prefix. Highlight unchanged tokenizer IDs→embeddings→logits→CE→sampling skeleton.
154	Can you finish this sentence?	Students complete: “A Transformer language model differs from our aabid MLP mainly because ...” Final answer: it builds each context representation by repeated, position-aware, causally masked, learned retrieval plus per-token MLP transformations.

Short-on-time routes

- Lecture 1: keep 1–2, 4–7, 9–17, 21–34, 40–50; move 3, 8, 18–20, 35–39, and 41–44 to handout review.
- Lecture 2: keep the bank role/message/addition story and pages 89–116 as one chain. On pages 112–113, inspect rows 3 and 6 instead of reading every matrix entry. Leave the appendix and the companion’s detailed gradient calculations for self-study. Keep the forward pass, generation, target loss, computational graph, autograd update, and causal mask in the class discussion.
- Lecture 3: keep 105–108, 112–115, 117–128, 130–142, 145, 147, 149–154.

Checks before teaching

1. The six aabid examples are consistent everywhere.
2. Every attention matrix states its row/column convention.
3. Every softmax identifies its normalization axis.
4. All Q/K/V and MHA shape equations compile and agree.
5. Future attention is zero after the causal softmax.
6. Training targets are shifted exactly one token.
7. Both handout and progressive-presentation PDFs compile.
8. All pages pass a visual overflow/contact-sheet inspection.
9. Technical diagrams remain native vector Typst/SVG. Generated illustrations support examples, with labels and equations added in Typst.
10. The companion PyTorch notebook executes from top to bottom and reproduces the tiny attention calculation used in the slides.

Companion material

- Two-to-four-line PyTorch chunks accompany lookups, projections, attention, prediction, generation, cross-entropy, backward, and optimizer updates.
- `lecture11/mega/bank_training.py` is the executable companion for the revised Part 2. Its JSON evidence contains full-precision tensors, gradients, generation, and before/after updates. Run it with `--check` in an environment with PyTorch. It retains manual-gradient and finite-difference checks for students who want to work through the derivatives after the main lecture.
- The complete executable companion is `lecture11/notebooks/from_characters_to_transformers.ipynb`.

Primary references

- Nipun Batra, *Next Token Prediction* (the aabid teaching anchor): <https://nipunbatra.github.io/ml-teaching/neural-networks/slides/next-token-prediction.pdf>
- MIT 6.S191, *Deep Sequence Modeling*: https://introtodeeplearning.com/slides/6S191_MIT_DeepLearning_L2.pdf
- Stanford CME 295, Lecture 1, *Transformer*: <https://www.youtube.com/watch?v=Ub3GoFaUcDs>
- 3Blue1Brown, *Attention in transformers, visually explained*: <https://www.3blue1brown.com/lessons/attention/>
- Vaswani et al., *Attention Is All You Need*: <https://arxiv.org/abs/1706.03762>